

CYFRIN · SECURITY REVIEW

Cyfrin

BattleChain Review.

A security review of BattleChain Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Cyfrin	BattleChain Review	Guardian	March 30 – May 12, 2026	May 19, 2026

§ 00

Contents.

33 pages — overview, confidence ranking, scope, 19 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 17 findings	P. 09
	Remediation Review · 2 findings	P. 29
§ 05	Disclaimer	P. 32
§ 06	About Guardian	P. 33

§ 01

Overview.

Guardian reviewed BattleChain Review across 2 review rounds. The final report includes 1 High, 1 Medium, 5 Low, 12 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	Cyfrin/battlechain-safe-harbor
CLIENT	METHODOLOGY
Cyfrin	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
BattleChain Review	f9e6e8500a0d
PREPARED BY	REMEDATION REVIEW COMMIT
Guardian	8a0c5ddd7790
AUDIT WINDOW	
March 30 – May 12, 2026	
FINAL REPORT	
May 19, 2026	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	1	0	0	0	1
Medium	1	0	0	0	1
Low	5	0	2	0	3
Informational	12	0	3	0	9
All severities	19	0	5	0	14

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed BattleChain Review in `Cyfrin/battlechain-safe-harbor`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Multiple Deployment Paths In BattleChainDeployer Will Always Fail	Logical Error	HIGH	RESOLVED	p. 9
M-01	diligenceRequirements Can Be Made Arbitrarily Burdensome During Commitment Window	Validation	MEDIUM	RESOLVED	p. 11
L-01	Stale promotionRequestTimestamp When Marked Corrupted	Logical Error	LOW	RESOLVED	p. 12
L-02	Aggregate Bounty Cap Can Be Introduced In Commitment Window	Validation	LOW	RESOLVED	p. 13
L-03	Favorable childContractScope Upgrade Is Blocked	Logical Error	LOW	ACKNOWLEDGED	p. 14
L-04	Switching From Aggregate Cap To Retainable Is Blocked Despite Being Favorable	Logical Error	LOW	RESOLVED	p. 15
I-01	Redundant Zero-address Checks In Initializers	Informational	INFORMATIONAL	RESOLVED	p. 16
I-02	Misleading Comment On requestUnderAttack	Informational	INFORMATIONAL	RESOLVED	p. 17
I-03	Agreement Mismatches Can Be Used to Deceive Whitehats	Documentation	INFORMATIONAL	RESOLVED	p. 18
I-04	Warning Regarding In Scope Contracts	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 20
I-05	Document Bond Forfeiture When Re-requesting	Documentation	INFORMATIONAL	RESOLVED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-06	Warning For Whitehats Regarding Mid-Attack Scope Changes	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-07	Old Agreements Break When The Factory Is Changed	Logical Error	INFORMATIONAL	RESOLVED	p. 24
I-08	finalizeState Emits Stale PRODUCTION Event	Informational	INFORMATIONAL	RESOLVED	p. 25
I-09	Duplicate Accounts Desynchronize Scope Cache	Validation	INFORMATIONAL	RESOLVED	p. 26
I-10	claimBond Reverts For Fee-only Deposits	Informational	INFORMATIONAL	RESOLVED	p. 27
I-11	Outdated Code Comment In AttackRegistry	Informational	INFORMATIONAL	RESOLVED	p. 28

REMEDIATION REVIEW

L-01	claimBond zero-bond bypasses state check	Logical Error	LOW	ACKNOWLEDGED	p. 29
I-01	Informational Note Regarding promotionRequestedTimestamp	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 31

HIGH **H-01**

RESOLVED

Multiple Deployment Paths In BattleChainDeployer Will Always Fail

CATEGORY Logical Error

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/BattleChainDeployer.sol#L33-L39>

REVIEW Main Review

DESCRIPTION

`BattleChainDeployer` overrides each `CreateX` deployment function to call `_registerDeployment(newContract)` after invoking `super`. However, multiple convenience overloads in `CreateX` are implemented by calling another overload (for example, `deployCreateAndInit(init, data, values)` calls `deployCreateAndInit(init, data, values, refundAddress)`, and `deployCreate2(initCode)` calls `deployCreate2(salt, initCode)`).

Because these internal calls are virtual, when `BattleChainDeployer` invokes `super.deployCreate2(initCode)`, the `CreateX` implementation internally calls the `deployCreate2(salt, initCode)` variant, which is also overridden by the `BattleChainDeployer`. That override then deploys the contract and registers the contract once. Control then returns to the original one, which proceeds to call `_registerDeployment` again, resulting in duplicate registration and reverts.

As a result, most deployment functions in `BattleChainDeployer` always revert, since any overload that internally calls another overload attempts to register the same contract twice and fails with the `ContractAlreadyRegistered` error.

The failing variants are as follows:

- `deployCreateAndInit(bytes memory initCode, bytes memory data, Values memory values)`
- `deployCreate2(bytes memory initCode)`
- `deployCreate2AndInit(bytes32 salt, bytes memory initCode, bytes memory data, Values memory values)`
- `deployCreate2AndInit(bytes memory initCode, bytes memory data, Values memory values, address refundAddress)`
- `deployCreate2AndInit(bytes memory initCode, bytes memory data, Values memory values)`
- `deployCreate2Clone(address implementation, bytes memory data)`
- `deployCreate3(bytes memory initCode)`
- `deployCreate3AndInit(bytes32 salt, bytes memory initCode, bytes memory data, Values memory values)`

H-01 Multiple Deployment Paths In BattleChainDeployer Will Always Fail

CONTINUED

- `deployCreate3AndInit(bytes memory initCode, bytes memory data, Values memory values, address refundAddress)`
- `deployCreate3AndInit(bytes memory initCode, bytes memory data, Values memory values)`

— RECOMMENDATION

Ensure that `_registerDeployment` is called only once per deployment. For overloads that delegate to another overload, consider removing the `_registerDeployment` call from the delegating wrapper to avoid duplicate registration.

PROOF OF CONCEPT <https://github.com/GuardianOrg/battlechain-safe-harbor-fuzz-1774884273263/pull/1>

— RESOLUTION

Status: Resolved.

MEDIUM **M-01**

RESOLVED

diligenceRequirements Can Be Made Arbitrarily Burdensome During Commitment Window

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/Agreement.sol#L273>

REVIEW Main Review

DESCRIPTION

The `BountyTerms` struct contains a `diligenceRequirements` string defining conditions whitehats must meet to claim bounties. The commitment-window checks in `setBountyTerms` guard five fields (`bountyPercentage`, `bountyCapUsd`, `aggregateBountyCapUsd`, `identity`, `retainable`) but not `diligenceRequirements`. During the commitment window, the owner can change this from empty to any arbitrary requirements, making bounty collection impossible.

As a result, whitehats who began work under no diligence requirements may be subjected to impossible conditions introduced mid-commitment, effectively preventing any bounty claims.

RECOMMENDATION

Block changes to `diligenceRequirements` during the commitment window.

RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

Stale promotionRequestTimestamp When Marked Corrupted

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/AttackRegistry.sol#L316
REVIEW	Main Review

DESCRIPTION

The `known-issues.md` document states: "If an attack is discovered during the 3-day promotion delay, the attack moderator should call `*cancelPromotion*` (returning to `*UNDER_ATTACK*`) and then `*markCorrupted*`". However, this behavior is not enforced in the contract.

`markCorrupted` can be called in both the `UNDER_ATTACK` and `PROMOTION_REQUESTED` states. If it is called directly during the `PROMOTION_REQUESTED` state without first cancelling the promotion, the agreement state becomes `CORRUPTED`, but the `promotionRequestedTimestamp` is not cleared. `getAgreementInfo` indicates that a promotion was requested and may be interpreted as still pending, even though the agreement has already been marked as corrupted.

Similarly, `promotionRequestedTimestamp` is not cleared when a terminal `PRODUCTION` state is reached. While this can serve as useful historical information about when the promotion was requested for that state, having a non-zero `promotionRequestedTimestamp` when the terminal state is `CORRUPTED` can be misleading and open to misinterpretation.

RECOMMENDATION

Consider enforcing a promotion cancellation before marking an agreement as corrupted, or clearing `promotionRequestedTimestamp` when the agreement reaches a terminal `CORRUPTED` state.

RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Aggregate Bounty Cap Can Be Introduced In Commitment Window

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/Agreement.sol#L291-L293>

REVIEW Main Review

DESCRIPTION

The commitment-window guard for `aggregateBountyCapUsed` in `Agreement.setBountyTerms` only prevents decreasing an existing non-zero cap. When the current value is 0 (no aggregate cap), `new_value < 0` is always false for `uint256`, so introducing a brand-new aggregate cap passes the check. Adding a cap where none existed is unfavorable to whitehats as it limits the total payout pool across all participants.

Whitehats who evaluated the agreement under no-cap terms face a retroactively imposed payout ceiling. If multiple whitehats are active, the aggregate cap may prevent each from receiving their full individual cap.

RECOMMENDATION

Do not allow setting a new cap during the commitment window if the previous cap was zero.

RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

Favorable childContractScope Upgrade Is Blocked

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/main/src/Agreement.sol
REVIEW	Main Review

— DESCRIPTION

During the commitment window, `addAccounts` is allowed because expanding scope is favorable to whitehats. However, there is no way to update the `childContractScope` of an existing account. For example, upgrading an account from `None` to `All` expands the attack surface, which is beneficial for whitehats. The only way to achieve this is to remove the account and re-add it with the new scope, but `removeAccounts` is blocked during the commitment window as an unfavorable change

```
function removeAccounts(string memory caip2ChainId, string[] memory accountAddresses) external  
onlyOwner {  
    // Cannot remove accounts during commitment window  
    if (block.timestamp < s_cantChangeUntil) {  
        revert Agreement__CannotReduceScopeDuringCommitment();  
    }  
    // ...  
}
```

As a result, a favorable scope expansion is unintentionally blocked.

— RECOMMENDATION

Consider allowing `childContractScope` upgrades (toward more inclusive scopes) for existing accounts during the commitment window.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Switching From Aggregate Cap To Retainable Is Blocked Despite Being Favorable

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/Agreement.sol#L463
REVIEW	Main Review

DESCRIPTION

`Agreement.setBountyTerms` enforces that `aggregateBountyCapUsd` cannot decrease during the commitment window. Separately, `_validateBountyTerms` requires `aggregateBountyCapUsd == 0` when `retainable = true`. These two constraints create a deadlock: an agreement with `retainable = false` and `aggregateBountyCapUsd > 0` can never switch to `retainable = true` during the commitment window, because doing so requires setting `aggregateBountyCapUsd` to 0 (a decrease).

Making the bounty retainable is strictly favorable to whitehats as they keep recovered funds instead of returning them. The commitment window is designed to allow favorable changes, but this particular favorable transition is blocked.

As a result, agreement owners who initially chose an aggregate bounty cap cannot upgrade to retainable bounties during the commitment window despite being favorable to whitehats.

RECOMMENDATION

Allow `aggregateBountyCapUsd` to decrease to 0 when switching to retainable in the same call.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-01

RESOLVED

Redundant Zero-address Checks In Initializers

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/AttackRegistry.sol#L131>, <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/AgreementFactory.sol#L70>

REVIEW Main Review

DESCRIPTION

The `initialize` functions across multiple contracts perform duplicate `address(0)` checks, as outlined below:

- `AttackRegistry.initialize` checks that `_initialOwner` and `_treasury` are not `address(0)` and reverts if they are. However, the subsequent internal calls to `__Ownable_init` and `_setTreasury` already perform these validations.
- `AgreementFactory.initialize` does the same for `_initialOwner` and reverts before calling `__Ownable_init(_initialOwner)`, even though `__Ownable_init` already reverts when the owner is zero.
- `BattleChainSafeHarborRegistry.initialize` checks whether `owner == address(0)` immediately before invoking `__Ownable_init_unchained(owner)`, which already enforces this constraint.

These checks are therefore redundant and can be safely removed to reduce code duplication.

RECOMMENDATION

Consider removing redundant `address(0)` validation checks where the downstream initializer already enforces the same constraint. While retaining these checks is functionally harmless, removing them can reduce duplication.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-02

RESOLVED

Misleading Comment On requestUnderAttack

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/AttackRegistry.sol#L222>

REVIEW Main Review

DESCRIPTION

The NatSpec comment on `requestUnderAttack` states that "the agreement owner must be the deployer of all contracts". However, the function actually checks the authorized owner for each contract, rather than the deployer.

Since deployers can delegate ownership via `authorizeAgreementOwner`, the deployer does not necessarily have to match the agreement owner, making the comment inaccurate.

RECOMMENDATION

Update the comment to state that "the agreement owner must be the authorized owner for all contracts".

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

RESOLVED

Agreement Mismatches Can Be Used to Deceive Whitehats

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/BattleChainSafeHarborRegistry.sol#L136-L139>

REVIEW Main Review

DESCRIPTION

Protocols adopting the BattleChain Safe Harbor deploy agreements and register them with both the `AttackRegistry` and the `BattleChainSafeHarborRegistry`. However, there is no enforcement or linkage between these two registries.

The BattleChain Safe Harbor documentation (<https://docs.battlechain.com/battlechain/explanation/safe-harbor#verifying-protection>) recommends this three-step check before attacking:

```
// 1. Agreement is valid
bool valid = safeHarborRegistry.isAgreementValid(agreementAddress);
// 2. Contract is attackable
bool attackable = attackRegistry.isTopLevelContractUnderAttack(contractAddress);
// 3. Contract is in scope
bool inScope = agreement.isContractInScope(contractAddress);
```

A protocol can satisfy all three booleans while pointing whitehats to an agreement that doesn't actually apply to the contract:

1. Deploy two agreements via the factory: Agreement A (generous terms, higher bounty, higher percentage) and Agreement B (stingier, low bounty, more strict requirements). Include the same contract addresses in both scopes.
2. Register stingier Agreement B with `AttackRegistry` (`requestUnderAttack`). This makes `attackRegistry.isTopLevelContractUnderAttack(contract)` return true and legally binds the contract to Agreement B.
3. Register favourable Agreement A with `BattleChainSafeHarborRegistry` (`adoptSafeHarbor`).

When whitehats fetch the agreement for that protocol via `BattleChainSafeHarborRegistry.getAgreement`, they receive Agreement A. When they then run the checklist using Agreement A:

- `safeHarborRegistry.isAgreementValid(A)` → `true`, because Agreement A was also factory-deployed.
- `attackRegistry.isTopLevelContractUnderAttack(contract)` → `true`, because Agreement B

I-03 Agreement Mismatches Can Be Used to Deceive Whitehats

CONTINUED

registered the contract.

- `AgreementA.isContractInScope(contract)` → `true`, because the early return in the constructor when adding contracts to the scope. While each contract can have only one agreement in the AttackRegistry, different agreements can include the same contract in their scope as long as they are not registered, due to no-op here.

All three checks succeed, yet the contract's actual governing agreement (Agreement B) has different bounty terms. Whitehats performing the recommended checks can still be misled into believing more generous terms apply and could even face legal consequences.

Note that the `adoptSafeHarbor` function includes the following comment:

```
/// @dev A user can add a malicious agreement here, this is ok
/// @dev but the attack registry can only use agreements deployed from the factory
```

This assumes that the added 'malicious' agreement is not deployed via the factory. However, protocols can deploy an agreement with malicious intent via the factory to pass checks and mislead whitehats.

— RECOMMENDATION

Consider enforcing that `adoptSafeHarbor` only accepts agreements already registered in the AttackRegistry, if an onchain fix is desired. This would prevent inconsistent agreements across registries. However, mismatches may still occur after the commitment window if the protocol updates the AttackRegistry but not the BattleChainSafeHarborRegistry.

Additionally, document that whitehats should verify the agreement directly in the AttackRegistry via `attackRegistry.getAgreementForContract(contractAddress)` for each contract, and confirm the agreement state using `attackRegistry.getAgreementState(agreementAddress)`

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-04

ACKNOWLEDGED

Warning Regarding In Scope Contracts

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

Safe Harbor protection applies only to the accounts explicitly listed within an agreement's scope (including any child contracts covered by `ChildContractScope`)

Complex exploits often traverse multiple contracts (e.g., entry points, routers, adapters, vaults). If a protocol's agreement includes only a subset (for example, just the vault), any intermediate contract not listed is technically outside the protection. This can create grey areas for both whitehats and protocols. For example:

- A vault is in scope, but the exploit requires an entry point in another contract that is not in scope. In this case, `isTopLevelContractUnderAttack` returns true for the vault but false for the entry-point contract, making it unclear whether attacking the vault is permitted.
- The entry point of an attack is within scope, but the contract holding the funds is not.
- Both the entry-point contract and the vault are in scope, but multiple intermediary contracts involved in the exploit path are not.

Everything is clear for whitehats when all contracts in an exploit path are either entirely in scope or out of scope. However, mixed cases introduce uncertainty and should be documented for both whitehats and protocols, as protocols may assume that including only fund-holding contracts is sufficient and exclude surrounding contracts.

RECOMMENDATION

For protocols, document that all relevant contracts should be included in these agreements, rather than only those holding funds.

For whitehats, clarify which contracts must be verified as in scope, whether that includes only the final contract holding funds, the entry-point contract, or the entire exploit chain.

I-04 Warning Regarding In Scope Contracts

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-05

RESOLVED

Document Bond Forfeiture When Re-requesting

CATEGORY Documentation

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/BondManager.sol#L165-L168>

REVIEW Main Review

DESCRIPTION

When `_collectFeeAndBond` runs, it checks for an existing bond deposit on the agreement and forfeits it if an unclaimed bond exists.

This is the intended behavior, as indicated by the `BondForfeited` event and the comment "Forfeit any existing unclaimed bond to keep `s_reservedByToken` accurate." However, this should be clearly documented, as agreement owners may not review the codebase directly.

RECOMMENDATION

Document the forfeiture behavior of unclaimed bonds. Alternatively, consider allowing previous depositors to claim at a later time, or directly transferring the existing bond to them.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Warning For Whitehats Regarding Mid-Attack Scope Changes

CATEGORY	Warning
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

After the minimum commitment window, the scope and agreement can be changed. The protocol may still be in the UNDER_ATTACK state while having passed the commitment window. During this period, a scope or agreement change may be recorded on-chain just before an attack transaction, either naturally or by the protocol front-running the attack with a scope change.

This may expose the whitehat to legal risk while performing the attack. To mitigate this, whitehats should perform scope and agreement validation and the attack atomically within the same transaction, ensuring there is no gap between validation and execution. This ensures that the attack either executes under a valid agreement or reverts if a scope change is mined before the attack transaction.

— RECOMMENDATION

Document this risk and warn users to perform scope validation and attacks atomically.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-07

RESOLVED

Old Agreements Break When The Factory Is Changed

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/main/src/AttackRegistry.sol#L695-L696
REVIEW	Main Review

DESCRIPTION

`registerContractForExistingAgreement` / `unregisterContractForExistingAgreement` only pass if `s_agreementFactory.isAgreementContract(msg.sender)` is true. The Agreement contract is `msg.sender`.

Old agreements were created by factory A and are only marked in A's `s_isAgreement`. If the registry owner changes to factory B via `setAgreementFactory`, those old addresses don't exist in B's mapping. The `isAgreementContract` check fails with `InvalidAgreement` revert.

Consequently, `addAccounts`, `removeAccounts`, `addOrSetChains`, `removeChains` (anything that syncs BattleChain scope) reverts for those agreements. Same for `syncNewContracts` when it hits the factory check on the agreement address.

RECOMMENDATION

Consider documenting this behavior, so that users are aware of it.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-08

RESOLVED

finalizeState Emits Stale PRODUCTION Event

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/main/src/AttackRegistry.sol#L867
REVIEW	Main Review

DESCRIPTION

When an agreement auto-promotes to PRODUCTION (either via the 14-day deadline without DAO approval, or via the 3-day promotion delay), the transition is computed only — `promoted` stays `false` in storage. Because `_revertIfLinkedToActiveAgreement` reads the computed state, contracts can immediately be re-linked to a new agreement, emitting `ContractRegistered(X, agreementB)`.

`finalizeState` is permissionless and has no expiry. If it is called after contracts have already been re-linked, it emits `AgreementStateChanged(agreementA, PRODUCTION)` out of order, after `ContractRegistered(X, agreementB)` has already appeared in the event stream.

Any off-chain system reconstructing "which agreement governed contract X at production time" will compute the wrong answer: it will appear that contract X was still under agreementA's umbrella when it graduated, when in reality X had already been reassigned to agreementB.

RECOMMENDATION

Consider requiring `finalizeState` to be called before contracts can be re-linked, or calling `finalizeState` in `_revertIfLinkedToActiveAgreement`.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-09

RESOLVED

Duplicate Accounts Desynchronize Scope Cache

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/Agreement.sol#L212>

REVIEW Main Review

DESCRIPTION

`Agreement.addAccounts` does not check for duplicate BattleChain account addresses (documented in NatSpec). The scope cache deduplicates via `s_battleChainScopeExists` in `_addToBattleChainScope`. When the same address is added twice to `s_accounts` but only once to the scope cache, a single `removeAccounts` call removes the cache entry entirely while one copy remains in `s_accounts`. The result is a state where `getChainAccounts` still includes the address but `getBattleChainScopeAddresses` and `isContractInScope` do not. Consequently, off-chain tools relying on `getDetails` show the contract in scope while on-chain queries show it out of scope.

RECOMMENDATION

Consider rejecting duplicate BattleChain additions, or document this behavior, as adding the same address twice constitutes an owner error.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-10

RESOLVED

claimBond Reverts For Fee-only Deposits

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/BondManager.sol#L92-L97>

REVIEW Main Review

DESCRIPTION

When `s_bondToken != address(0)` but `s_verifiedBondAmount = 0` (or `s_unverifiedBondAmount = 0`), `BondManager._collectFeeAndBond` creates a `BondDeposit` with `bondAmount = 0`. The depositor pays only a fee. When the agreement later reaches `PRODUCTION`, the depositor calls `claimBond`. The lazy-mark block is gated on `deposit.bondAmount > 0` and is skipped. `_markBondClaimable` also returns early for `bondAmount == 0`. So `bondClaimable` is never set to `true`, and `claimBond` permanently reverts with `BondNotYetClaimable`.

While claiming a bond with a zero amount is unnecessary and has no impact on funds, off-chain automation tools that call `claimBond` after going into production will fail.

RECOMMENDATION

Consider handling zero-bond deposits as an early success and marking them as claimable within the lazy-marking block, thereby allowing zero-bond claims when necessary.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-11**

RESOLVED

Outdated Code Comment In AttackRegistry

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1774884269008/blob/f9e6e8500a0d50b613ef5d37b8631663411b3030/src/AttackRegistry.sol#L17
REVIEW	Main Review

DESCRIPTION

The comment in the `AttackRegistry` contract states that "NEW_DEPLOYMENT is for contracts deployed via BattleChainDeployer, and NOT_DEPLOYED is for external deployments." However, this is incorrect, as the `NEW_DEPLOYMENT` state is not tied to `BattleChainDeployer`.

`_getAgreementState` returns `NEW_DEPLOYMENT` only when `info.isRegistered == true` and `attackRequested == false`; however, all registration paths set `attackRequested` to true.

All unregistered agreements are treated as `NOT_DEPLOYED`, regardless of whether they are externally deployed or deployed via `BattleChainDeployer`.

RECOMMENDATION

Update the outdated comment and the README to reflect the current behavior.

RESOLUTION

Status: Resolved.

LOW **L-01**

ACKNOWLEDGED

claimBond zero-bond bypasses state check

CATEGORY	Logical Error
LOCATION	src/BondManager.sol:91-97
REVIEW	Remediation Review

DESCRIPTION

The I-10 fix added a short-circuit at the top of `claimBond` so fee-only (zero-bond) deposits no longer permanently revert. As written, the short-circuit runs before the lazy-mark / PRODUCTION check that gates non-zero claims:

```
if (deposit.bondAmount == 0) {
    emit BondClaimed(agreementAddress, deposit.depositor, 0);
    deposit.claimed = true;
    return;
}
// Lazy-mark for time-based promotions that bypassed explicit marking
if (!deposit.bondClaimable) {
    if (_getAgreementState(agreementAddress) == IAttackRegistry.ContractState.PRODUCTION) {
        emit BondClaimable(agreementAddress, deposit.bondAmount);
        deposit.bondClaimable = true;
    }
}
if (!deposit.bondClaimable) {
    revert BondManager__BondNotYetClaimable(agreementAddress);
}
```

For non-zero bonds the depositor must wait until `PRODUCTION` (or until the agreement is rejected / corrupted) before `claimBond` succeeds. For zero-bond deposits there is no such gate. The depositor can call `claimBond` immediately after `_collectFeeAndBond`, while the agreement is still in `ATTACK_REQUESTED` or `UNDER_ATTACK`, and the deposit gets flipped to `claimed = true`.

RECOMMENDATION

Move the short-circuit inside the existing PRODUCTION lazy-mark gate, matching the I-10 recommendation.

L-01 claimBond zero-bond bypasses state check

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Informational Note Regarding promotionRequestedTimestamp

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/battlechain-safe-harbor-team1-1778247843968/blob/8a0c5ddd7790f5d835fddee69b86f811fb4610fb/src/AttackRegistry.sol#L329
REVIEW	Remediation Review

DESCRIPTION

`promotionRequestedTimestamp` is now cleared on explicit terminal paths, so the original L-01 issue has been fixed. Multiple comments in the codebase now state that a non-zero timestamp indicates a promotion is pending.

- In known-issues.md: "preserving the invariant that a non-zero timestamp means promotion is currently pending"
- In AttackRegistry.sol: "// Maintain invariant: promotionRequestedTimestamp != 0 iff promotion currently pending"

However, this is not entirely accurate, since `promotionRequestedTimestamp` can remain non-zero while the state is `PRODUCTION` after `PROMOTION_DELAY` has elapsed but before `finalizeState` has been called.

RECOMMENDATION

Update the comments to reflect that a non-zero `promotionRequestedTimestamp` does not necessarily mean the promotion is still pending, and may also represent the period where the state has effectively transitioned to `PRODUCTION` but `finalizeState` has not yet been called.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.